ModeClass / ModeManager 使い方ガイド

対象ファイル: main/head.hpp, main/ModeManager.ino

2026 年 10 月 30 日

目次

1 概要 2

2 API リファレンス 2

2.1 ModeClass(各モードの基底クラス) 2

2.2 ModeManager(グローバル変数 MM) 3

3 新しいモードを追加する手順 3

4 使用例 3

4.1 例 1: 一定時間たったら次のモードへ (時間で遷移) 3

4.2 例 2: センサの値で遷移する 4

4.3 例 3: 切り替わった時の初期化に whenChanged() を使う 4

4.4 例 4: 割り込んで, 終わったら元のモードに戻る 5

4.5 例 5:loop() から優先して切り替える 5

5 注意点・よくあるミス 6

6 コピペ用テンプレート 7

6.1 モードクラスのテンプレート 7

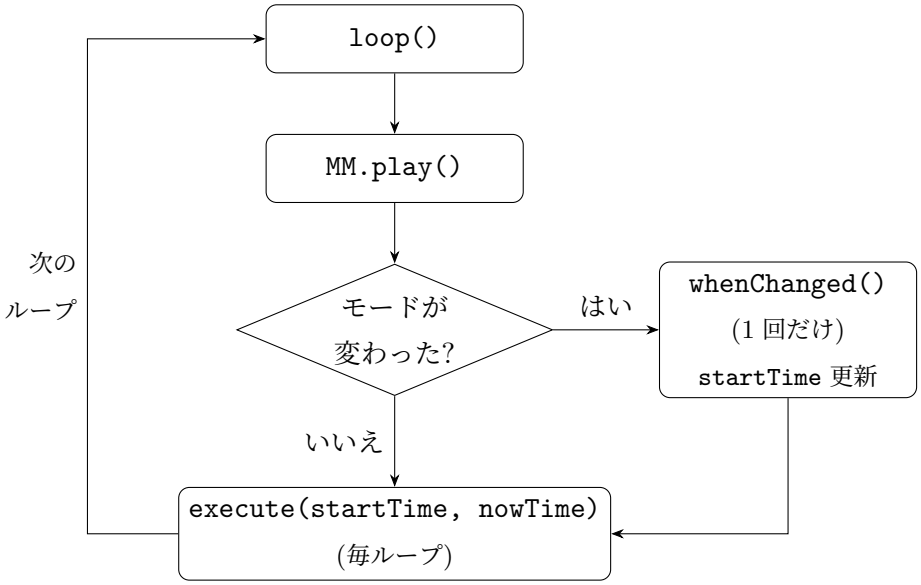
6.2 main.ino に追加するもの 8

6.3 head.hpp にモードを追加する場合 8

1　概要

ロボットの動作は「モード」ごとに分けて書きます. 1 つのモードにつき 1 つのクラスを作り, そのクラスは ModeClass を継承します. どのモードを今動かすかは, グローバル変数 MM(ModeManager のインスタンス) が管理します.

- **Mode**(enum class): モードの名前.Mode::Forward のように書く. 数字では書かない.
- **ModeClass**: 各モードの処理を書くための基底クラス.
- **ModeManager MM**: 登録されたモードクラスを保持し, 現在のモードの処理を呼び出す.



2　API リファレンス

2.1　ModeClass(各モードの基底クラス)

メンバ	説明
Mode myMode	このクラスが担当するモード. **コンストラクタで必ず設定する**.
Mode getMode()	myMode を返す.MM.set() の中で使われる.
void whenChanged()	【純粋仮想関数】他のモードから**このモードに切り替わった直後に 1 回だけ**呼ばれる. 初期化・音を鳴らす・変数リセットなど.
void execute(long startTime, long nowTime)	【純粋仮想関数】このモードの間, **毎ループ**呼ばれる.startTime はこのモードに入った時刻 [ms],nowTime は現在時刻 [ms](どちらも millis() の値).

whenChanged() と execute() は純粋仮想関数なので, **両方とも必ず実装**してください (中身が空でもよい).

2.2 ModeManager(グローバル変数 MM)

メソッド	説明
`bool set(ModeClass *target)`	モードクラスを登録する. コンストラクタ内で `MM.set(this)` と書く.
`Mode play()`	現在のモードの `whenChanged()/execute()` を呼ぶ.`loop()` で毎回呼ぶ (既に `main.ino` に書いてある). 現在のモードを返す. クラスが未登録なら `Mode::Error` を返して何もしない.
`void changeMode(Mode next)`	通常のモード変更. **実際に切り替わるのは次の `play()`**.
`void changeModeForce(Mode next)`	優先度の高いモード変更. これを呼ぶと, 次の `play()` までの間 `changeMode()` が無視される. 現在と同じモードを指定した場合は何もしない.
`Mode getModePrev()`	1 つ前のモードを返す. 割り込み的なモード (黒線復帰など) から元に戻るときに使う.

3 新しいモードを追加する手順

手順 1. `head.hpp` の `enum class Mode` にモード名を追加する (既にあれば不要). `Num` より前に書くこと.

手順 2. `ModeClass` を継承したクラスを書く (6 節のテンプレートをコピペ).

- コンストラクタで `this->myMode = Mode::` **なんとか**`;` と `MM.set(this);`
- `whenChanged()` と `execute()` を実装

手順 3. `main.ino` でそのクラスのグローバル変数を 1 つ作る.
例:`Pro1 pro1;` ← これを書かないと登録されず, そのモードは動かない.

手順 4. 他のモードから `MM.changeMode(Mode::` **なんとか**`);` で遷移させる.

注意:enum の途中に追加すると番号がずれる

`mode_G` は `(int)` の数値として Processing に送られ, 画面にも数値で表示されます. `enum` の途中にモードを追加すると, それ以降のモードの番号が 1 つずつずれるので, Processing 側で番号を見ている場合は合わせて確認してください.

4 使用例

4.1 例 1: 一定時間たったら次のモードへ (時間で遷移)

`startTime` と `nowTime` の差が「このモードに入ってからの経過時間」です. `delay()` で待たずに経過時間で判定すると, 待っている間もセンサやボタンの処理が止まりません.

```
class A : public ModeClass
{
```

```
public:
  A(){
    this->myMode = Mode::Mode0_A;
    MM.set(this);
  }

  void whenChanged() override{
  }

  void execute(long startTime , long nowTime) override{
    if (nowTime - startTime >= 5000){  // このモードに入って5秒たったら
      MM.changeMode(Mode::Forward);
    }
  }
};
```

4.2 例 2: センサの値で遷移する

色センサの結果はグローバル変数 color_G に入っています. changeMode() を呼んだ後も execute() の残りは実行されるので, **すぐ return;** しておくと安全です.

```
void execute(long startTime , long nowTime) override{
  Color c = (Color)color_G;

  if (c == Color::Red || c == Color::Blue){
    motors.setSpeeds(0, 0);
    MM.changeMode(Mode::Resume);
    return;  // 下のsetSpeedsを実行させない
  }

  motors.setSpeeds(120, 120);
}
```

4.3 例 3: 切り替わった時の初期化に whenChanged() を使う

モードクラスのオブジェクトはプログラム終了まで残り続けるので, メンバ変数の値は前回そのモードにいた時のまま残ります. モードに入るたびにリセットしたい変数は whenChanged() で初期化します.

```
class Search : public ModeClass
{
private:
  int count = 0;

public:
  Search(){
    this->myMode = Mode::Rotate;
    MM.set(this);
  }
```

```
  void whenChanged() override{
    count = 0;          // モードに入るたびに0から数え直す
    tone(3, 440);       // 切り替わったことを音で知らせる
  }

  void execute(long startTime , long nowTime) override{
    if (nowTime - startTime >= 200){
      noTone(3);
    }
    count++;
  }
};
```

4.4 例 4: 割り込んで, 終わったら元のモードに戻る

黒線復帰のように「どのモードからでも割り込んで, 終わったら元に戻る」モードは getModePrev() を使います.

```
void execute(long startTime , long nowTime) override{
  motors.setSpeeds(-200, -200);  // 少し下がる
  delay(100);
  motors.setSpeeds(200, -200);   // 回転
  delay(800);

  MM.changeMode(MM.getModePrev());  // 割り込む前のモードに戻る
}
```

4.5 例 5:loop() から優先して切り替える

ボタンが押されたらどのモードでもリセットモードにする, のような最優先の遷移は changeModeForce() を使います. 同じループ内で他の場所が changeMode() を呼んでも上書きされません.

```
void loop()
{
  getColor();
  color_G = (uint8_t)Nearest_Neighbor();

  if (button.isPressed() == true){
    MM.changeModeForce(Mode::Reset);  // これが最優先
  }

  checkBlackLine();       // ここでchangeModeされても無視される
  mode_G = MM.play();
  receive();
  delay(50);
}
```

5　注意点・よくあるミス

動かないときに確認すること

1. **main.ino にグローバル変数を書いたか.** 書いていないとクラスが登録されず, そのモードに遷移した瞬間に play() が Mode::Error を返し続けて何も動かなくなります.
2. **コンストラクタで myMode を設定してから MM.set(this) しているか.** 順番が逆だと間違ったモードで登録されます.
3. **同じ Mode を担当するクラスを 2 つ作っていないか.** 2 つ目の登録は無視されます (先に登録された方が使われる).
4. **changeMode() した直後に切り替わると思っていないか.** 切り替わるのは次の MM.play() です.execute() の残りの処理はそのまま実行されます.
5. **changeModeForce() の後に changeMode() していないか.** 次の play() までは changeMode() が無視されます.

知っておくと便利

- 起動直後の最初のモード (Mode::Mode0_A) では whenChanged() は呼ばれません.
- 今と同じモードに changeMode() すると,whenChanged() がもう一度呼ばれ startTime もリセットされます (changeModeForce() では何も起きません).
- execute() の中で長い delay() を使うと, その間 loop() が止まり, 色センサの更新やボタンの検出もされません. なるべく経過時間 nowTime - startTime で判定してください.
- loop() の最後に delay(50) があるので,execute() が呼ばれる間隔はおよそ 50 ms 以上です.

6　コピペ用テンプレート

6.1　モードクラスのテンプレート

以下をコピーして, ☆ MyMode ☆ をクラス名に,Mode::XXXX を担当するモードに,Mode::YYYY を次のモードに置き換えてください. TestClass.hpp などの既存ファイルに追記するか, 新しい .hpp ファイルを作って main.ino で #include します.

```
#pragma once
#include "head.hpp"

/*****************************************
 * ☆MyMode☆ / XXXX : このモードでやること
 * 補足 :
 *****************************************/
class ☆MyMode☆ : public ModeClass
{
private:
  // このモードだけで使う変数・定数
  static const int SPEED = 200;

public:
  ☆MyMode☆(){
    this->myMode = Mode::XXXX;  // 担当するモード
    MM.set(this);               // マネージャーに登録
  }

  /* このモードに切り替わった直後に1回だけ呼ばれる */
  void whenChanged() override{
    MySerial.println("☆MyMode☆");
  }

  /* このモードの間，毎ループ呼ばれる */
  void execute(long startTime , long nowTime) override{
    long elapsed = nowTime - startTime;  // このモードに入ってからの時間[ms]

    // 終了条件を満たしたら次のモードへ
    if (elapsed >= 3000){
      motors.setSpeeds(0, 0);
      MM.changeMode(Mode::YYYY);
      return;
    }

    // 毎ループやる処理
    motors.setSpeeds(SPEED, SPEED);
  }
};
```

6.2 main.ino に追加するもの

```
#include "☆MyMode☆Class.hpp"  // 新しいファイルを作った場合のみ

☆MyMode☆ myModeObj;            // グローバル変数として1つ作る（これで登録される）
```

6.3 head.hpp にモードを追加する場合

```
enum class Mode
{
  /* ... 既存のモード ... */
  Reset,    //リセットモードに遷移
  XXXX,     //新しいモードの説明  ← Numより前に追加

  /* エラーが起きた時に */
  Error,
  /* モード数 */
  Num
};
```